CSCI 6100 Operating Systems and Systems Architecture
Programming Project

Please refer to the syllabus for the academic honesty policy. Your project is expected to be an individual effort. It is also expected to be neat and clearly organized. You must submit a softcopy of **the source codes in a WORD document, your original source codes (e.g., *.c, *.java files) in a zip file, and PowerPoint slides explaining how each component of your program interacts with one another** by the specified due date and time. You can find the link for your submission on D2L.

This project is due: November 18, 2025, 11:59 PM
Total points: 10

A university computer science department has a teaching assistant (TA) who helps undergraduate students with their programming assignments. The TA's office is rather small and has room for only one desk with a chair and computer. There are three chairs in the hallway outside the office where students can sit and wait if the TA is currently helping another student. When there are no students who need help during office hours, the TA sits at the desk and takes a nap. If a student arrives during office hours and finds the TA sleeping, the student must awaken the TA to ask for help. If a student arrives and finds the TA currently helping another student, the student sits on one of the chairs in the hallway and waits. If no chairs are available, the student will come back at a later time. For simplicity we assume each student asks for helps at most two times. Using any synchronization mechanisms and a choice of your own language, implement a solution that coordinates the activities of the TA and the students. Details for this assignment are provided below.

The Students and the TA

Using threads, begin by creating *n* students. Each will run as a separate thread. The TA will run as a separate thread as well. Student threads will alternate between programming for a period of time and seeking help from the TA. If the TA is available, they will obtain help. Otherwise, they will either sit on a chair in the hallway or, if no chairs are available, will seek help at a later time (random period of time). If a student arrives and notices that the TA is sleeping, the student must notify the TA using a semaphore. When the TA finishes helping a student, the TA must check to see if there are students waiting for help in the hallway. If so, the TA must help each of these students in turn. If no students are

present, the TA may return to napping. To simulate students programming in students threads, and the TA providing help to a student in the TA thread, the appropriate threads should sleep (by invoking sleep()) for a random period of time.
For simplicity, each student thread terminates after getting help twice from the TA. After all students' threads terminate, the program cancels the TA thread and then entire program terminates.

Task: You are required to implement a solution to the problem above using concepts covered in class, especially threads and synchronization mechanisms (e.g., mutexes or semaphores). You may use any programming language. After implementation, create a PowerPoint that explains your approach and shows the key code that implements mutual exclusion.